Streaming Mixture-of-Experts Inference on a 32 GiB Desktop: A Measurement Study with Negative Results

Michael Gasperini — Vastloom / TheStreamCode

Working paper, August 2026. Not peer-reviewed.

Abstract

Context. Open-weight Mixture-of-Experts (MoE) checkpoints have outgrown the memory of the machines most people own. The common responses are to buy memory, rent a datacenter, or memory-map the checkpoint and let the operating system's page cache absorb the difference.

Objective. We ask whether a purpose-built streaming runtime can execute an 82.5 GB quantized MoE checkpoint — DeepSeek V4 Flash 0731 — on a 31 GiB desktop with a 12 GB GPU, correctly and without rendering the host unresponsive; and, if such an engine is slower than an existing one, what evidence would justify building it at all.

Method. We built Vastloom, a C++20 runtime that reserves host RAM, VRAM, staging and scratch before allocating and refuses work it cannot afford. We measured it against llama.cpp on the same host, checkpoint and prompt, under a declared reporting protocol that treats machine state as part of every measurement.

Results. The engine produces the external oracle's greedy output at full depth. We establish a closed-world bandwidth budget of 8.76 GB of weight traffic per decoded token, of which 80% is deterministic always-active weight: the per-layer trunk plus six global tensors. Delegating matrix-vector products to ggml moved the CPU path from 27,925 to 4,282 ms/token; residency, overlap and lifetime changes took it to 2,587 ms/token on the IQ1_S profile, and the current IQ2_XXS execution path measures 3.2–3.5 s/token. llama.cpp decodes the same class of workload at 302 ms/token when the host is idle and 662 ms/token under contention. We report eight optimization directions closed with measurements, and a class of five measurement instruments that could not have failed and therefore could not inform.

Conclusions. The contribution is not a faster engine. It is a documented map of which ideas survive contact with a 32 GiB machine, which instruments lie, and where a safety-first streaming runtime remains distinguishable from memory-mapping and hoping. The one remaining differentiator we identify — speculative verification designed for a streaming cost model — could not be measured on this host without breaching a pre-declared safety floor, and is already implemented upstream at the draft-format level.

Keywords: mixture-of-experts, local inference, memory hierarchy, NVMe streaming, speculative decoding, negative results, measurement methodology.

Venue fit. This is a systems experience-and-negative-results report, not a methods paper claiming a new state of the art. Appropriate homes are workshops at MLSys, SOSP or OSDI, or a preprint in cs.DC / cs.LG. It is not a NeurIPS/ICML methods submission.

---

1. Introduction

A decode step of a sparse MoE model at batch size 1 is bandwidth-bound: it reads a large, mostly fixed set of weights and performs comparatively little arithmetic on them. When the weights exceed host memory, the read has to come from somewhere else on every token. The prevailing engineering answer is `mmap`: map the checkpoint, let the page cache hold what it can, and accept that the operating system decides what stays resident.

That answer has two properties worth questioning. First, the page cache is a least-recently-used policy with no knowledge of the model's structure, so it cannot distinguish a weight that every token needs from one that a router selected once. Second, it provides no admission control: a machine can be driven into swap-thrashing by a workload that was accepted without anyone asking whether it fits.

Vastloom was built to test whether an explicit, model-aware alternative is worth its complexity. It is the instrument of this study rather than its claim.

1.1 Research question

> Can a consumer PC with 31 GiB of RAM and a 12 GB GPU execute DeepSeek V4 Flash 0731 > locally and correctly, without risking host stability — and if a purpose-built engine is > slower than an existing one on that machine, what would have to be true for it to be worth > building?

1.2 Contributions

1. **A closed-world bandwidth budget** for this checkpoint at batch 1: 8.76 GB of weight traffic per token, 80% of it deterministic always-active weight — the per-layer trunk plus six global tensors (§4). Any strategy that optimizes only routed-expert traffic is bounded at roughly 1.25× end-to-end even in the limit of completely free experts, and at roughly 1.11× for a halving of that traffic.
2. **An external-oracle correctness result** (§5). Two implementations written from a single reading of the Hyper-Connection equations agreed with each other and were both wrong. No internal test could have separated them; an independent implementation did, on the first comparison.
3. **A catalogue of five measurement instruments that could not fail** (§8), each of which was used to justify engineering effort before the defect was noticed. We consider this the most transferable result in the paper.
4. **A kernel-versus-streaming decomposition** (§6). After delegating compute to ggml, matvec microbenchmarks measure 100–199 GB/s while the end-to-end engine delivers 2.9–3.4 GB/s on the same profile. The residual gap to llama.cpp is I/O structure, not missing SIMD.
5. **Eight optimization directions closed with measurements** (§7), recorded so they are not retried without new reasons.
6. **A pre-registered falsification attempt** of a streaming-speculation thesis (§9), its failure for host-capacity reasons, and the independent finding that llama.cpp already ships the relevant draft support.

1.3 Non-claims

We do not claim competitive throughput, a novel quantization format, a novel scheduling algorithm, or that Vastloom should replace llama.cpp on this machine today. Where a number in this paper is favourable, §11 states the conditions under which it is not.

---

2. Background

2.1 The checkpoint

DeepSeek V4 Flash 0731 in Unsloth's GGUF distribution: 43 main layers, 256 routed experts per layer with top-6 selection, hash routing in the first three layers and learned routing in the remaining forty. The catalog partitions exactly: 1,328 physical tensors become 34,223 logical descriptors accounting for 82,533,892,444 encoded bytes — 1,193 per-layer trunk tensors (6,390,088,008 bytes), six global tensors (596,000,788 bytes) and 129 packed expert containers holding 11,008 three-matrix expert bundles.

Two quantization profiles appear in this paper and **must not be compared to each other**:

- `UD-IQ1_S` (76.9 GB payload), used for the measurements in §6.1–6.3. This store was

subsequently deleted and is no longer an execution path (§2.2).

- `UD-IQ2_XXS` (84.6 GB store), the current execution path, used in §6.4–6.5.

2.2 Vastloom: the instrument

Vastloom is a C++20 runtime with a two-plane structure. The control plane owns hardware discovery, capacity planning, telemetry and the **Safety Governor**; the data plane owns bounded I/O, staging buffers, kernels and residency transitions. No data-plane worker may allocate outside the Governor.

The Governor reserves before allocating. A command whose exact resident cost exceeds the live safe budget — computed per run as `min(half of physical RAM, available RAM − 8 GiB)` — is refused in advance with exit code 75. That refusal is a designed outcome, not a failure, and the test harness reports it as *skipped* rather than *passed*, so a refused contract can never be mistaken for a verified one.

A product decision constrains the whole study: **IQ1 profiles are not an execution path.** Following the argument that 2-bit quantization of this model family retains usable quality while 1-bit does not [8], the IQ1_S store was deleted. Measurements taken on it are retained as history and are never presented as current product quality.

2.3 Terminology

Term	Meaning in this paper
Trunk	The always-active per-layer weights: attention, normalization, router, shared expert. Every token reads all of them.
Globals	The six non-layer tensors read on every token path: embedding, LM head, output norm and the three output Hyper-Connection tensors.
Always-active weight	Trunk plus globals — everything a decode step reads regardless of routing. 6.99 GB on this checkpoint.
Expert bundle	The three matrices (gate, up, down) of one routed expert. Indivisible: a partial bundle cannot be computed.
Working set	Weight bytes a single decode step must traverse, independent of where they come from.
Logical bytes	Bytes the engine requested. Distinct from process-visible I/O and from physical device reads; we report the three separately.
Residency	Explicit placement of a buffer in a tier (NVMe / RAM / VRAM) under a Governor lease, with eviction blocked while a consumer holds a use lease.
Trunk window	The number of layer trunks kept resident simultaneously; `--trunk-window N`.

Units. GB denotes 10^9 bytes and GiB denotes 2^{30} bytes throughout. Byte counts derived from the checkpoint catalog are exact; memory figures reported by the operating system are as-observed.

---

3. Methodology

3.1 Hardware and software

Component	Value
CPU	AMD Ryzen 9 7900X, 24 hardware threads
RAM	31.1 GiB usable

GPU	NVIDIA RTX 4070 Super, 12 GB, compute capability 8.9
OS	Windows 11
Vastloom	MSVC 19.44, Release, warnings as errors; optional ggml backend
ggml	0.19.0, commit `30bf868`, `/arch:AVX512`, statically linked
llama.cpp	build `b1-84e908c`, MSVC, CPU-only, AVX-512, 12 threads

The same pinned ggml commit serves as the parity oracle for the storage decoders and, when `--ggml-compute` is enabled, as the compute backend. §11 treats the consequences of that overlap.

3.2 Workload

All timed generation figures use one prompt: an Italian capital-city question under the checkpoint's chat template, greedy decoding (`--temp 0 --top-k 1`), seed 0, 12 new tokens, context 512, no warmup. Under a correct engine the continuation contains **`Roma`**; that string is the correctness check applied to every timed run.

Prompt length is fixed because it is not a free variable: prefill scales with it, and a 28-character prompt and a 70-character prompt on the same configuration measured 8,405 and 20,682 ms/token respectively. Comparing runs with different prompts has produced at least one wrong constant in this project's history.

3.3 Instrumentation

Every run emits a JSON record. Byte accounting is kept in four separate counters because conflating them has previously produced a false conclusion (§8, item 4):

- **logical bytes** — what the engine requested from the catalog;
- **process-visible I/O** — `GetProcessIoCounters` on Windows, `/proc/self/io` on Linux;
- **physical device reads** — `IOCTL_DISK_PERFORMANCE` mapped to the checkpoint's `PhysicalDriveN`, or `/sys/dev/block/.../stat` on Linux, with whole-device scope declared;
- **PCIe uploads** — `gpu_uploaded_bytes`, counted at the transfer, never inferred.

Host memory safety is sampled via `Win32_PerfFormattedData_PerfOS_Memory.AvailableBytes`. `FreePhysicalMemory` is explicitly *not* used: under `mmap` the pages of mapped files are reclaimable standby memory, which `FreePhysicalMemory` excludes, producing premature aborts.

3.4 Reporting rules

A token time is not a property of an engine until the prompt, cache state, free RAM, quantization profile and whether the GPU path was live are all stated. This rule is not stylistic. On this host, byte-identical llama.cpp binaries with identical flags measured 1.51 tok/s and 3.06 tok/s **on the same morning**; prompt eval was unchanged (650.97 vs 646.55 ms/token) while decode halved (662.08 vs 326.29 ms/token). Batch-1 decode is bandwidth-bound, and page-cache warmth decides which figure a run receives. We therefore treat that pair as one fact about contention, not as two engine versions, and we do not compare numbers taken in different machine states.

3.5 Safety protocol

The host owner's standing constraint is that the machine must remain interactive. Every experiment declares a floor for available memory *before* it starts and is killed on breach. No run uses `--no-mmap` or `--mlock`; one run executes at a time. Aborts under this protocol are reported as results (§9.3), not omitted.

3.6 Statistical treatment

Measurements are single runs unless stated otherwise. This is a limitation, declared rather than smoothed: run-to-run variance on identical configurations was measured at ±9%, so differences below

roughly 10% are not treated as signal anywhere in this paper. Where a change produced a difference inside that band we say so explicitly (§6.3). No confidence intervals are reported because the repetition counts to support them were not collected.

---

4. The bandwidth budget of a token

Because the routed selection is top-6 of 256 and the always-active weight is by definition read in full, the per-token weight traffic of one decode step is a closed-world quantity derived from the catalog rather than an estimate.

Table 1. Weight traffic per decoded token (batch 1, derived from the checkpoint catalog).

Component	GB / token	Share
Layer trunks (always active, 1,193 tensors over 43 layers)	6.39	73%
Global tensors (LM head, output norm, Hyper-Connection, embedding)	0.60	7%
Selected experts (top-6 of 256, 258 bundles)	1.77	20%
Total	**8.76**	100%

The rows are the exact catalog byte counts: 6,390,088,008 trunk bytes, 596,000,788 global bytes and 1,770,651,648 expert bytes (258 complete bundles), totaling 8,756,740,444 bytes. The executed full-depth forward pass read exactly 1,770,651,648 expert bytes on its no-cache pass — the derivation matches observation to the byte. The always-active two rows together (6.99 GB) are the 80% that every optimization below must confront. The embedding is read once per prompt rather than per token, so steady-state decode is closer to 8.46 GB/token; 8.76 is the conservative first-pass figure used throughout.

Scope: this budget is derived from the `UD-IQ1_S` *catalog. The trunk and global rows are identical in the* `UD-IQ2_XXS` *profile (its preflight reports the same 6,390,088,008 and 596,000,788 bytes), but its experts are larger, so the current execution path's per-token total is higher; it has not been recomputed, and no per-token bandwidth figure in this paper should be applied to that profile.*

Correction note: an earlier version of this budget reported 8.00 GB/token. That figure was an addition error — 6.39 + 2.37 was written as 8.00, and the 2.37 row conflated the experts with the globals. All quantities derived from the budget are recomputed here against the corrected total.

At 1.8 tok/s an engine must therefore sustain 15.8 GB/s of *effective* weight traffic. A dual-channel desktop memory bus supplies roughly 30–50 GB/s, placing the batch-1 ceiling for any engine on this hardware between roughly 3.4 and 5.7 tok/s if every byte fetched is useful and the bus is saturated. llama.cpp's idle 3.06–3.31 tok/s already delivers 27–29 GB/s of effective weight traffic — the share of a desktop memory bus that well-written SIMD kernels actually extract.

4.1 Implications

Two consequences follow directly, and both were counter-intuitive enough to have misdirected work before they were computed:

1. **Halving expert traffic saves 0.89 GB/token, bounding end-to-end gain at about 1.11×**, before any quality cost; even in the limit of completely free experts the bound is 1.25×. The always-active weight is the bill. An engine that optimizes expert caching alone is competing for a fifth of the problem.
2. **Speculative verification changes the denominator rather than the constant.** Verifying K accepted tokens in one pass traverses the 6.99 GB of always-active weight once instead of K times. At $K = 7$ that weight amortizes to 1.00 GB per accepted token. Expert bytes do not amortize away — each accepted token still selects its own six experts per layer — but the draft's exact routing lets the engine deduplicate

the window's union (token-to-token recurrence is 26%, §6.5) and read it as one sequential batch, bringing expert traffic from 1.77 toward roughly 1.4 GB per accepted token. The total is between 2.4 GB (measured recurrence) and 2.8 GB (no deduplication) per accepted token — a factor of 3.2–3.7, and a change in *what is paid for*. §9 treats this as the study's central thesis and reports why it could not be measured here.

A separate observation constrains where the bytes actually come from. On a warm host, direct (unbuffered) expert I/O measured **3× slower** than buffered reads, because 88% of logical bytes were already in the page cache and only ~0.98 GB per pass reached the SSD. **Storage is not the bottleneck on this machine once the working set has been touched.** The page cache is also not a controlled tier — it is precisely why the same binary halves its decode rate under contention (§3.4).

---

5. Correctness before performance

5.1 The external oracle

For most of this project's history, correctness was defended by internal fixtures and by a Python reference implementation written from the model's published equations. Both passed. llama.cpp, built from source against the same GGUF and given the identical twelve token ids, disagreed on the answer.

The disagreement was ours. The Hyper-Connection mixing matrix is stored `[source][destination]`, and Sinkhorn's final normalization is over sources, so the contraction that sums to one runs over `source` for fixed `destination`. Vastloom summed the other way — and so did the Python operator reference, because it had been written from the same reading of the same equations. Two implementations confirmed each other for an entire milestone. Every layer multiplied the residual by slightly more than one, and the hidden-state RMS grew from 6.8 to 4,064 across forty-three layers.

After the index was corrected, the eight strongest logits match the oracle in order and the model produces the expected continuation.

The methodological point generalizes beyond this bug. Internal consistency between two artifacts derived from a common source is not evidence. An independent implementation is a different kind of instrument, and this project found no substitute for it.

5.2 What bit-exactness means here

We claim bit-exactness for **storage decoders** — each GGML type (Q8_0, Q5_K, Q6_K, Q4_K, IQ1_S, IQ2_S, IQ2_XXS, IQ3_XXS, MXFP4, BF16, F32) is gated against pinned ggml row output — and we explicitly do not claim it for accumulation. Vastloom's reference path accumulates in `double` where ggml accumulates in `float`; logit differences of 0.01–0.06 are expected by construction and are not decoder defects. Conflating the two would make every parity gate either impossible to pass or impossible to fail.

---

6. Results: where the time goes

6.1 Delegating compute to ggml

Vastloom's scalar reference kernels exist so that every storage type has a decoder that can fail a test; they are not a production inner loop. An opt-in backend hands matrix-vector products to the pinned ggml commit that the parity oracles already name. The Governor reserves the backend's pool before anything allocates; unsupported shapes fall back call by call to the scalar path; a backend error is an error, never a silent substitution.

Table 2. Scalar reference kernels vs. delegated ggml backend. ***IQ1_S profile, 43 layers, 12 tokens,*** `--cpu-only`***, trunk window 2, synchronous experts.***

Metric	Scalar	ggml backend
ms / token	27,925	**4,282**
Expert compute (total)	231.3 s	9.1 s
Matvecs accelerated	—	39,181
Fallbacks to scalar	—	276
Peak host commit	—	1.79 GB
Generated ids	reference	identical

The parity gate for this path was built so it can fail: Q8_0 and F32 against the scalar reference within accumulation-plus-input-quantization noise (0.084 observed against 0.2 allowed; F32 at 9.5×10^{-7}), plus a corruption probe comparing clean and corrupted backends with the noise removed.

6.2 Microbenchmarks: the kernels stop being the story

Table 3. Matvec throughput, same binary, cache-hot unless noted.

Kernel	Shape	Throughput	Per-call overhead
Q8_0	2048 × 2048	199 GB/s	0.02 ms
Q8_0	32768 × 2048 (71 MB, streaming)	100 GB/s	—
IQ1_S	2048 × 2048	16 GB/s	—
IQ1_S	scalar reference, same shape	0.19 GB/s	—
IQ1_S	cold experts, in situ	~3.2 GB/s	—

Re-run note (2026-08-19): the same gate re-executed on the same host measured Q8_0 at 167 GB/s cache-hot and 95 GB/s streaming, IQ1_S at 14 GB/s and the scalar IQ1_S reference at 0.18 GB/s, with the parity checks passing again (0.084 against the 0.2 tolerance; F32 at 9.5×10^{-7}). The recorded table values reproduce within machine-state variance (§3.6).

AVX-512 is compiled in. There is no missing instruction-set object to link, and no kernel in the profile is within an order of magnitude of being the constraint at 8.76 GB/token.

6.3 Residency and overlap are path-conditioned

Holding more trunks resident had previously been measured as *slower*, and was recorded as a closed direction. That result was taken on the hybrid GPU path, where host commit for device memory (§10) competes with the accelerator's own ceiling. Repeating the sweep with the GPU removed reversed it.

Table 4. Trunk window sweep, CPU-only ggml path, IQ1_S profile, 12 tokens.

Configuration	ms / token	Trunk reloads	Measured overlap
window 2, synchronous experts	4,816	494	0
window 8, synchronous experts	4,541	428	0
window 43, synchronous experts	3,446	43	0
window 43, asynchronous experts	**3,063**	43	5,017 ms

A later change to the same profile — releasing each layer's experts after it finishes rather than when the next layer begins, and charging one selected set in the preflight instead of two layers' worth — took this configuration to **2,587 ms/token** at a peak commit of 7.46 GB against a 7.83 GB budget, with identical ids. That is the best IQ1_S figure recorded, and the last one: the profile was retired immediately afterwards

(§6.4).

Window 43 retains every layer trunk after the first pass, reducing trunk traffic from 68.4 GB to 5.94 GB across the run at a peak commit of 7.09 GB — inside the live budget. Two asynchronous workers then submit selected expert misses after routing, and the shared expert, which is trunk-resident, computes while those loads are in flight. Generated ids are unchanged throughout.

A negative result inside the same experiment. Hash-routed layers select experts by token-id table lookup, so their loads can legally begin before the gate matvec. Doing so moved wall time from 3,063 to 3,003 ms/token — inside the ±9% variance band, therefore *not* a result. Only three of forty-three layers are hash-routed; the learned-router layers cannot prefetch from the token id, because the router reads the post-attention hidden state.

The lesson is that a negative result is path-conditioned. "A wider trunk window is slower" was true, on the path where it was measured, and false one configuration away. We now record the path alongside every closed direction in §7.

6.4 Current position on the IQ2_XXS execution path

The IQ1_S store was deleted (§2.2); the figures below are the current product path and are **not comparable** to Tables 2 and 4.

Table 5. IQ2_XXS, 43 layers, `--cpu-only --ggml-compute --trunk-window 43`**.**

Configuration	ms / token	Note
2 async workers	3,317	first IQ2 generate on this host
4 async workers	3,198	overlap 4.9 → 7.8 s
4 workers, colder page cache	3,488	42.7 GB device reads vs 23.8

The engine produces the oracle-correct greedy string throughout, at approximately 0.3 tok/s. Expert loading remains most of a ~32 s generate even after four workers raised measured overlap from 4.9 to 7.8 s. IQ2 experts are larger than IQ1 experts, so the move from 2.6 s/token (IQ1) to 3.2 s/token (IQ2) is a change of workload, not a regression — and per §4's scope note, the 8.76 GB/token budget does not transfer to this profile either.

6.5 Expert recurrence: a real effect that does not pay

Token-to-token, **26% of selected expert identities recur** (1,070 of 4,102 on the pinned prompt). Persisting the last six experts per layer across decode reduced expert traffic from 31.2 GB to 24.6 GB — the effect is real and the size is as predicted.

It still made the run slower. Affording that persistence required shrinking the trunk window from 43 to 9, which raised trunk reloads from 43 to 383 and took the run from 3.27 to **6.48 s/token**. Persistence is therefore enabled only when window 43 already fits the live budget.

This is Table 1 asserting itself: an expert-cache win purchased with trunk residency loses, because the trunk is four times the traffic.

6.6 Where the residual factor of ten sits

On the IQ1_S profile — the only one for which both a per-token byte count and a like-for-like comparison exist — the engine delivers 2.9 GB/s of effective weight traffic at 3,063 ms/token, or 3.4 GB/s at the 2,587 ms/token best. llama.cpp's idle 3.3 tok/s on the same profile corresponds to 27–29 GB/s. We attribute the difference to three causes, in the order their evidence supports:

1. **Governed expert I/O after the shared-expert overlap is exhausted** — acquisition is serial in expert-id order.
2. **IQ decode cost** — 16 GB/s cache-hot and ~3.2 GB/s on cold experts, against Q8_0's 100+.

3. **The `mmap` path itself** — llama.cpp consumes file-backed pages in place and lets the page cache be the cache, with no copy into engine-owned buffers.

A model-aware engine *should* be able to beat an LRU page cache under contention; §3.4 shows that cache losing half its throughput on an unmodified binary. That differentiator is not visible at 3 s/token, and becomes testable only once the streaming path is within a small factor of `mmap` on an idle host.

---

7. Directions closed with measurements

Each row below has a commit and a recorded measurement. They are listed so they are not retried without a new reason — and, per §6.3, each is annotated with the path on which it was measured, because that qualification turned out to matter.

Table 6. Optimization directions rejected, with the evidence and its scope.

Direction	Result	Path
Load one tensor instead of one layer	Identical 65.3 GB of traffic, 19% slower	hybrid GPU
Recency eviction in VRAM under a cyclic scan	`trunk_reloads` 473 before and after	hybrid GPU
Expert deduplication / shared basis	Mean cosine 0.0005 across two quantizations and two depths; residual 0.936 vs 0.9375 expected for independent vectors — experts are orthogonal	format-independent
Interleaved Q8_0 layout (coalescing, order-preserving)	21% slower: 8.27 vs 6.81 s/token. Repacking costs one pass per upload; at reuse factor 1.03 it never amortizes	hybrid GPU; coupled to residency, not refuted in general
Direct (unbuffered) expert I/O	3× slower; 88% of logical bytes served from page cache	CPU
Removing the GPU entirely	28.5 vs 6.88 s/token — 4.1× worse	pre-ggml scalar kernels
Wider trunk window / pinning / all 43 resident	No gain, 25% slower, or 4× worse	hybrid GPU **only** — reversed on CPU-only ggml (§6.3)
Raising the accelerator ceiling 4.33 → 5.20 GB	No measurable change: uploads 8.54 GB/pass, reuse 1.03, ~20,300 evictions, at both ceilings	hybrid GPU

The last row is a step function, not a slope: both ceilings sit below the 6.39 GB trunk, and under a cyclic scan a cache smaller than the working set evicts everything before its next use regardless of size. 5.20 GB buys exactly what 1.05 GB buys.

---

8. Instruments that could not fail

This section reports the study's most expensive methodological defect. In five separate instances we built a check that passed for structural reasons, then used it to direct engineering effort. Each is stated with the mechanism that made falsification impossible.

1. **Dyadic parity gates.** A fixed scale of 0.25, a delta of 0.125 and inputs drawn from $k/256$ make every product exactly representable, so `float` and `double` accumulation agree bit for bit. Two kernels accumulating in the wrong precision passed. *A reported error of exactly zero deserves suspicion until the arithmetic is known not to be degenerate.*

2. **Cost attribution by subtraction.** Time not attributed to experts was labelled "trunk" and reported at ~50%. Direct measurement put it near 9% on that profile. A milestone followed the wrong number.
3. **A hidden-state norm sampled after the normalization that fixes it.** The probe reported 19.72 at every depth, which reads as reassuring stability. Sampled *before* the norm, the same state ran from 6.8 to 4,064 — the defect of §5.1, invisible to the instrument pointed at it.
4. **PCIe traffic assumed equal to logical bytes.** Resident matrices are not re-uploaded. The correct instruments are the transfer-side counters, kept separate from logical byte accounting (§3.3).
5. **A reserve measured in the regime where it is not needed.** Unmodelled host allocation was measured at 0.23 GB under a small GPU ceiling and used to argue that a 3 GiB reserve was thirteen times larger than necessary. Cutting it to 1 GiB passed preflight (`fits: true`, 2.75 GB required against an 11.24 GB budget) and then failed in the tile loader, because the accelerator's own commit had displaced buffers that nothing counted.

The fifth is the subtlest and the one we consider generalizable. The quantity was not fixed by construction; it was measured under conditions where the effect that bounds it does not occur. **Before trusting a number, establish the conditions under which it was taken** — and prefer instruments that have been demonstrated to fail on a deliberately broken input.

---

9. Speculative decoding for a streaming cost model

9.1 Thesis

Verifying *K* draft tokens in a single pass amortizes the 6.99 GB of always-active weight across *K* accepted tokens (§4.1). The draft additionally yields *exact* expert identities for those tokens — routing is a deterministic function of the draft's own hidden states — letting the engine deduplicate the window's expert union and convert scattered per-layer expert reads into one sequential batch. A sequential-versus-scattered read figure of 1.77 against 0.57 GB/s was recorded for this host during planning; unlike every other number in this paper it has no run record in the repository, so it is reported as an order-of-magnitude indication rather than as evidence.

Crucially, **the benefit is asymmetric**: it scales with how much an engine pays to re-read the always-active weight. For an engine whose page cache already holds it, amortization returns little; for one built around streaming it is the difference between 8.76 GB per token and 2.4–2.8 GB per accepted token.

9.2 Pre-registered verdict

The verdict table below was written and committed **before** the measurement, and is reproduced unmodified. Pre-registration is the reason this section can report an inconvenient outcome without reinterpretation.

Table 7. Pre-declared outcomes and their conclusions.

Observation	Conclusion
An existing engine reaches ≥ 4 tok/s with speculation	Thesis already implemented; no project
Speculation present but < 2.5 tok/s	Thesis holds; build the streaming verifier
No engine can load the draft	Field open; thesis holds
Everything below 1.8 tok/s even without speculation	The limit is the machine

9.3 Outcome: not measurable within the host's safe envelope

The baseline was obtained cleanly: llama.cpp at 3.06 tok/s (`llama-completion`) and 3.31 tok/s (`llama-server`), correct output, host idle.

The speculative configuration (DSpark Q8_0 draft, 10.1 GB, `--spec-draft-n-max 7`) was attempted three times and aborted three times under the §3.5 protocol:

1. Killed at 2.8 GiB `FreePhysicalMemory` — subsequently identified as the wrong metric (§3.3), since it excludes reclaimable standby pages.
2. Guard switched to Available Bytes with a 2 GiB floor. Trajectory: 22.0 → 17.5 → 16.3 → 6.7 → 1.1 GiB in 8 seconds; killed. The draft had not yet begun loading.
3. Target page cache pre-warmed by a complete baseline run immediately prior. Identical trajectory; killed at 1.1 GiB.

Initialization faults roughly 21 GiB of mapped pages in ~8 s (76.9 GB target plus 10.1 GB draft mapped onto a 31.1 GiB host), and the Windows trimmer does not reclaim quickly enough to hold the floor while the draft still has to load. **The blocker is host capacity, not code.**

9.4 What was established regardless

llama.cpp `b1-84e908c` ships first-class support for this draft family: a dedicated implementation with an explicit DSpark mode, Markov head, anchor-first block layout, block-size metadata reading, registration of both `draft-dflash` and `draft-dspark` speculation types, and wiring of the draft context to the target context.

The thesis is therefore *not* "no one has speculative decoding for this model." It is "no one has designed it for a streaming cost model," and that narrower claim remains untested on this host. Testing it requires a machine that can absorb the initialization spike — on the order of 48 GiB of free RAM.

Verdict. None of the four pre-declared rows applies exactly, and we record that rather than forcing a fit. The closest is *the limit is the machine*, with the caveat that the idle baseline already exceeds the 1.8 tok/s that row assumed.

---

10. Safety as a first-class result

Several measurements in this paper exist only because allocation is reserved before it is attempted and refused with a distinct exit code that the test harness records as *skipped* rather than *passed*.

One coupling turned out to be load-bearing and had been assumed rather than measured: on Windows, WDDM backs device allocations with host commit charge.

Table 8. Host commit charge per device allocation (measured).

Device allocation	Host commit delta	Marginal ratio
1 GB	1.25 GB	1.25
2 GB	2.25 GB	1.13
4 GB	4.26 GB	1.06
6 GB	6.26 GB	1.04

A fixed ~0.25 GB for the CUDA context, with the marginal cost converging on 1.00. Until this was charged to both budgets, a VRAM reservation passed a check that never asked whether the host could back it, and the failure surfaced wherever host memory was next requested — a preflight returning `fits: true`, followed by `bad_alloc` in the tile loader mid-pass. After booking the same bytes against both budgets, the same command completes: the accelerator self-limits to a 3.34 GB ceiling, peak commit 6.52 GB inside budget, zero declined matvecs, and a refusal now arrives where it can be handled by declining one matrix rather than by aborting a forward pass.

The same table closes a question this project reopened repeatedly. **Trunk residency on the GPU path is unreachable on this hardware**: 6.39 GB of resident weights cost an equal amount of host commit, on top

of a working set already near 2.8 GB, against a budget near 11 GB.

An engine that can lock a desktop is not a democratization project. We count these refusals as successful outcomes.

---

11. Threats to validity

11.1 Construct validity

The oracle and the backend share an implementation. The storage decoders are gated against pinned ggml output; llama.cpp — the end-to-end correctness oracle — is built on ggml; and since §6.1 the compute backend *is* ggml. A defect in ggml's decoders would therefore be invisible to all three instruments simultaneously. The Hyper-Connection defect (§5.1) was caught precisely because llama.cpp was independent of Vastloom's *graph*; no comparable independence exists for the storage layer. Establishing it would require a decoder implemented from the format specification by a third party.

"Correct" is defined by greedy string equality plus logit ordering, on a single prompt. This is a weak correctness criterion: it detects gross defects and would not detect a small systematic bias that leaves the argmax intact. No perplexity or downstream-task evaluation was performed, so this paper says nothing about quality at IQ2 versus higher precision.

11.2 Internal validity

Single runs. Timed figures are single measurements against a ±9% observed variance (§3.6); differences below ~10% are not treated as signal, and the §6.3 prefetch result is reported as null for exactly this reason.

The page cache is uncontrolled. Cache state is reported when known, but no run flushed the global cache — the safety protocol forbids the disruption. The 3,198 vs 3,488 ms/token pair in Table 5 differs mainly in cache warmth (23.8 vs 42.7 GB of device reads), which bounds how finely any of these figures can be read.

Instrument scope. Device-level counters have whole-device or partition scope and may include unrelated traffic. Windows exposes no per-process physical-byte counter, so process-visible transfers and physical reads are different quantities and are reported separately.

11.3 External validity

One host, one checkpoint, one operating system. Every conclusion is conditional on 31 GiB of RAM, 12 GB of VRAM, Windows/WDDM and this checkpoint's 80/20 trunk-to-expert split. A model with a different sparsity ratio inverts §4.1's arithmetic. Linux is tested for correctness but not for these performance figures; macOS and Metal are unvalidated.

Profile changes mid-study. §6.1–6.3 are IQ1_S; §6.4–6.5 are IQ2_XXS. They are reported separately and never compared, but that also means no single configuration was measured end-to-end across the whole optimization sequence.

The comparison engine was measured on the older profile. llama.cpp's 302 ms/token is an IQ1_S figure. We do not have a like-for-like IQ2_XXS llama.cpp baseline, so the "≈10×" gap should be read as an order of magnitude, not a coefficient.

11.4 Conclusion validity

The central negative claim of §9 is an *absence* of measurement, not a measurement of absence. We report the thesis as untested on this host rather than as unsupported.

---

12. Related work

Sparse expert models. Conditional computation via sparsely gated MoE layers [1] and its scaling to large-scale pretraining [2] established the top-*k*-of-*N* structure this checkpoint uses. Our study concerns execution of such a model under a memory constraint, not its training or architecture.

Offloaded and heterogeneous-tier inference. ZeRO-Infinity [3] and FlexGen [4] demonstrate that NVMe and host DRAM can serve as inference tiers under explicit scheduling, primarily for throughput-oriented batched workloads. Our regime is the opposite corner: batch 1, latency per token, and a hard constraint that the interactive host must not degrade. Work on offloaded MoE inference for consumer GPUs [5] targets the same expert-caching question and reports locality that our §6.5 measurement partially corroborates (26% recurrence) and partially undercuts (the trunk, not the experts, dominates this checkpoint's traffic).

Bit allocation by tensor role. RotaryQuant [6] targets the same problem class from the quantization side rather than the scheduling side: it assigns bit-widths by layer type (4-bit dense, 2-bit routed experts, 8-bit shared expert), compresses the KV cache with a transform followed by 3-bit quantization, pages experts to disk under memory pressure, and reports 9-19 tok/s fitting a 120B model in 32 GB on a fused Metal pipeline. The regimes differ in a way that matters for reading those numbers against ours: their compressed model fits the machine and pages under pressure, whereas our 82.5 GB checkpoint is 2.7x the host's RAM and streams on every token, on Windows/WDDM over PCIe rather than on Apple unified memory. §13 takes up the implication their bit allocation has for our Table 1.

Speculative decoding. Speculative decoding [7] and speculative sampling accelerate batch-1 decoding by verifying a draft's proposals in one target pass, and are standard where a draft model exists. Our §4.1 contribution is the observation that the *value* of that amortization is a function of how much the engine pays to re-read weights — an angle that matters for streaming engines and is invisible to an engine whose weights are resident.

Production engines. llama.cpp and ggml are the production CPU/GPU baseline, and in this study serve as both correctness oracle and compute backend (§11.1). DeepSeekV4SSD demonstrates that expert layout and ready-order compute can make a large sparse model interactive on 64 GiB of unified memory; it is not a Windows / 32 GiB / 12 GB-VRAM result, and the difference in host envelope is exactly what this paper measures. ds4 [8] motivated our quantization floor. Colibrì and kimi-k3-in-c are additional independent implementations consulted as engineering references.

We claim novelty for none of these ideas. We claim a measured intersection: this checkpoint, this memory envelope, this safety constraint, and this set of shortcuts that did not survive.

---

13. Discussion

What the evidence supports.

- A governed streaming runtime can execute an 82.5 GB sparse checkpoint correctly on a

31 GiB host, with the host remaining interactive throughout.

- The cost of a token on this checkpoint is 8.76 GB, four-fifths of it deterministic

always-active weight, and that single number bounds most optimization strategies before they are attempted.

- Several intuitively appealing optimizations are false on this host, with tables (§7) — and

at least one of them is false only on the path where it was first measured (§6.3).

- Some measurement instruments are structurally incapable of falsifying the hypothesis they

are pointed at; identifying them is itself a result (§8).

- After delegating compute, the residual deficit is streaming structure, not arithmetic.
- Safety refusals are load-bearing engineering outcomes, not error handling (§10).

What the evidence does not support.

- That Vastloom is faster than existing engines. It is not, on this host, on any profile measured.
- That NVMe streaming beats a warm page cache. The direct-I/O result (§4.1) points the other way on a warm host.
- That speculative decoding helps a streaming engine. This is the thesis; it is untested (§9.3).
- That expert sparsity within a top-6 routed, ~2-bit expert leaves usable work unexploited. No quality-per-byte curve was measured.

The largest lever our own arithmetic identifies is not in the engine. Table 1 says the always-active weight — the trunk plus the globals — is 80% of per-token traffic. In this checkpoint that weight is also where the highest-precision formats sit — Q8_0 attention, Q5_K/Q6_K shared expert, Q4_K embedding and LM head — while the routed experts that make up the other 20% are already near 2 bits. Bit-width is therefore allocated in inverse proportion to the traffic it generates. RotaryQuant [6] assigns 4 bits to dense layers for the same class of hardware; arithmetic on Table 1, not a measurement, puts a comparable halving of the always-active formats at roughly 3 GB off an 8.76 GB token — larger than any scheduling result in this paper.

Two qualifications keep this from being a recommendation. We did not measure it: no 4-bit-dense profile of this checkpoint was executed, and no quality-per-bit curve was collected, so the perplexity cost is unknown here. And the runtime cannot reach it — Vastloom treats published weights as immutable and does not requantize, so this lever belongs to whoever produces the distribution, not to the engine that streams it. That is worth stating plainly: the biggest number in our own budget is one our instrument cannot move.

Practical recommendation. For the goal "run this model on this PC today," llama.cpp answers it. For the goal "a streaming engine that is *different* on memory-poor hosts," the next evidence is not another kernel: it is an idle-host streaming path within a small factor of `mmap`, then a contention experiment where a model-aware policy can beat an LRU page cache, and — on a host that can hold target plus draft — a speculative-verification measurement that returns a number.

---

14. Conclusion

The interesting object produced by this project is not a faster matvec. It is a map: of a token's 8.76 GB bill, of instruments that cannot fail, of eight ideas that died with numbers, of a correctness defect that only an independent implementation could reveal, and of a differentiating thesis that remains untested because 31 GiB is not enough to load a draft model safely alongside its target.

We publish the map because the negative results in it were expensive to obtain, and because the methodological failures — five checks that could not say no — are, in our assessment, more transferable than any of the performance numbers.

Reproducibility and artifacts

Code. Vastloom is Apache-2.0. Real-weight tests are opt-in through `VASTLOOM_*` CMake paths; a green `ctest` without those paths does **not** reproduce any generation table in this paper. `docs/VALIDATION_MATRIX.md` maps each claim to the test that defends it, states when that test runs, and lists what must exist on the machine for it to run at all.

Data. Commands, binaries, prompts and per-run JSON records for the principal tables are recorded in `RESUME.md`, `CHANGELOG.md`, `docs/research/PHASE_A1_SPECULATION_FEASIBILITY.md` and the M9 validation reports under `docs/reports/`.

Weights. Not redistributed; they remain subject to their original licenses.

Reproduction constraints. Do not compare runs differing in prompt length, quantization profile, `--cpu-only`, `--trunk-window`, or free RAM. Report the skip count alongside the pass count for any `ctest` run: a refused contract is not a passing one.

Known gaps in this study's own evidence. Repetition counts sufficient for confidence intervals were not collected; no like-for-like IQ2_XXS baseline exists for the comparison engine; and no quality evaluation was performed at any quantization level.

Acknowledgements

llama.cpp and ggml (MIT) serve as this study's external oracle and, latterly, its compute backend. The 0731 GGUF and DSpark draft are Unsloth / DeepSeek distributions. The host owner required that no experiment render the machine unusable; several results reported here are aborts that respected that constraint.

References

1. N. Shazeer et al. *Outrageously Large Neural Networks: The Sparsely-Gated Mixture-of-Experts Layer.* ICLR, 2017.
2. W. Fedus, B. Zoph, N. Shazeer. *Switch Transformers: Scaling to Trillion Parameter Models with Simple and Efficient Sparsity.* JMLR, 2022.
3. S. Rajbhandari et al. *ZeRO-Infinity: Breaking the GPU Memory Wall for Extreme Scale Deep Learning.* SC, 2021.
4. Y. Sheng et al. *FlexGen: High-Throughput Generative Inference of Large Language Models with a Single GPU.* ICML, 2023.
5. A. Eliseev, D. Mazur. *Fast Inference of Mixture-of-Experts Language Models with Offloading.* Preprint, 2023.
6. A. Lui, M. Elsaied, N. P. Savani. *RotaryQuant: Fitting 120B MoE Models on Consumer Hardware via Fused Compressed-Space Attention.* Preprint, 8 August 2026. arXiv:2608.08081
7. Y. Leviathan, M. Kalman, Y. Matias. *Fast Inference from Transformers via Speculative Decoding.* ICML, 2023.
8. S. Sanfilippo. *ds4.* https://github.com/antirez/ds4
9. ggml-org. *llama.cpp* and *ggml.* https://github.com/ggml-org/llama.cpp, https://github.com/ggml-org/ggml
10. yanun0323. *DeepSeekV4SSD.* https://github.com/yanun0323/deepseek_ssd
11. JustVugg. *Colibrì.* https://github.com/JustVugg/colibri
12. FareedKhan-dev. *kimi-k3-in-c.* https://github.com/FareedKhan-dev/kimi-k3-in-c
13. Unsloth. *DeepSeek-V4-Flash-0731-GGUF.* https://huggingface.co/unsloth/DeepSeek-V4-Flash-0731-GGUF

External literature is cited by title and venue. Where this paper's own artifacts are the evidence, they are named by repository path in the Reproducibility section above.